Used Car Deal Hunter

Complete Study Guide

A Weeks 6-8 LLM Engineering capstone: estimate a fair used-car price, then hunt a held-out test set for listings priced well below that estimate.

- Package: car-deal-hunter v0.1.0 (Python >= 3.11)
- UI: Gradio (Estimate, Hunt deals, Metrics)
- Models: Random Forest + Chroma RAG (kNN / optional LLM) + optional QLoRA specialist
- Data: Yoad22/craigslist-used-cars-eda (CC-BY-4.0, from Austin Reese Kaggle scrape)
- Constraints: no OpenAI fine-tuning, no GPT APIs, no Modal GPU hosting, no live marketplace scraping

How to use this PDF: read sections 1-4 for the big picture, then 5-12 for every file. Section 16 is exam-style questions. Keep the architecture diagram in section 3 next to the code.

1. What this project is

Used Car Deal Hunter is a local, $0-API rebuild of a course 'price estimator + deal scanner' system. The original course path used paid OpenAI fine-tuning, GPT APIs, and Modal GPUs. This project keeps the same ideas (curate data, train a baseline, retrieve similar listings, ensemble agents, scan a feed, show a UI) but substitutes free public data, scikit-learn, local Chroma, optional Ollama, and optional Google Colab QLoRA.

What the app does for a user

1. Estimate: you type year, make, miles, condition, fuel, drivetrain, body, and state. The app rebuilds listing text and returns a fair-price estimate plus a per-agent breakdown.
2. Hunt deals: the scanner walks a shuffled held-out TEST split (not live Craigslist). For each listing it computes discount = estimate - asking_price. If discount >= threshold (default $1,500), it is an alert.
3. Metrics: a table of MAE, RMSE, and R-squared for the median baseline vs the Random Forest, measured on 400 test listings.

Hard product rule
The scanner does not scrape Craigslist, Facebook Marketplace, or any live site. The 'market feed' is test.parquet. That keeps the demo legal, reproducible, and free.

Course mapping (memorize this table)

Course week	This project
Week 6 data curation	car_hunter/curator.py + listing_text_from_row
Week 6 RF / eval	scripts/train_baselines.py, Metrics tab
Week 7 QLoRA	notebooks/qlora_colab.ipynb on free Colab T4
Week 8 specialist on Modal	local adapter folder or Ollama / Groq / Gemini
Week 8 RAG + ensemble	agents/rag.py, ensemble.py, vectorstore.py
Week 8 scanner / planner / UI	scanner.py, planner.py, messenger.py, app.py

2. Mental model in one paragraph

A used-car listing is a structured row (year, make, miles, ...) rewritten into English sentences. A Random Forest predicts price from those fields. A vector index stores train-set descriptions and prices; at query time the five nearest neighbors' prices are averaged (and optionally an LLM reads those neighbors). A specialist LLM can add a third guess if Ollama/Groq/Gemini or a QLoRA adapter is present. The ensemble blends those guesses. A planner walks test listings, scores discount, and a messenger appends JSONL alerts. Gradio is only a thin UI over DealHunter.

Discount definition

```
discount = ensemble_fair_price - asking_price
A deal exists when discount >= DEAL_THRESHOLD  (default 1500)
```

If the forest+RAG think a Honda is worth $18,000 and the test listing asks $15,200, discount is $2,800, which beats $1,500, so it is a deal. If they think it is worth $15,000 and it asks $16,000, discount is negative: not a deal.

3. Architecture

```
Hugging Face dataset  Yoad22/craigslist-used-cars-eda
      |
      v
 curator.clean + 80/10/10 split  ->  data/train|val|test.parquet     (Week 6)
      |
      |  listing_text_from_row  (reconstruct ad text from structured fields)
      |
      +----> RandomForestRegressor pipeline  ->  data/random_forest.joblib
      |
      +----> MiniLM embeddings in Chroma       ->  data/chroma/  (collection used_cars)
      |
      +----> optional SFT jsonl + Colab QLoRA  ->  data/qlora_adapter/
                  |
                  v
            EnsembleAgent
        0.65*RF + 0.35*RAG     OR     0.5*RF + 0.3*RAG + 0.2*Specialist
                  |
    Scanner (test feed) -> Planner.score -> Messenger (alerts.jsonl) -> Gradio
```

Runtime object graph

DealHunter creates one CarIndex (SentenceTransformer + PersistentClient) and one PlanningAgent. PlanningAgent owns ScannerAgent, EnsembleAgent, and MessengerAgent. EnsembleAgent owns ForestAgent, RagAgent, and SpecialistAgent. ForestAgent loads the joblib pipeline once. RagAgent reuses the shared CarIndex. SpecialistAgent checks for adapter_config.json, else LLM_PROVIDER.

Lazy UI vs eager models

app.py keeps hunter = None until the first Estimate or Hunt click. Metrics markdown is built at import time from metrics.json and does not load Chroma. The first user action constructs DealHunter, which loads MiniLM and opens Chroma and will fail if the collection is empty.

4. Repository map

Path	Role

app.py	Gradio UI: Estimate, Hunt, Metrics
pyproject.toml	Package name, deps, hatch wheel of car_hunter
car_hunter/config.py	Paths, filters, LLM env, deal threshold
car_hunter/curator.py	Download, clean, split, load_splits
car_hunter/listings.py	Pydantic models + text reconstruction
car_hunter/baselines.py	RF pipeline, median pricer
car_hunter/evaluator.py	MAE/RMSE/R2, parse_price
car_hunter/vectorstore.py	CarIndex: encode, add, query
car_hunter/llm.py	Ollama / Groq / Gemini / off
car_hunter/framework.py	DealHunter facade
car_hunter/agents/*	Colored log agents (see sec. 10)
scripts/prepare_data.py	CLI around curator.prepare
scripts/train_baselines.py	Fit RF, write metrics.json
scripts/build_index.py	Embed train listings into Chroma
scripts/export_sft.py	Write *_sft.jsonl for Colab
notebooks/qlora_colab.ipynb	Qwen2.5-1.5B QLoRA on T4
.env.example	LLM_PROVIDER and keys

What is gitignored (do not publish)

- .env (secrets)
- data/*.parquet, *.joblib, *_sft.jsonl, alerts.jsonl
- data/chroma/, data/qlora_adapter/, data/hf_cache/
- .venv/, __pycache__/

metrics.json is not ignored, so the Metrics tab can ship with the repo. The parquet, forest, and chroma folders are local artifacts you rebuild with the three scripts.

5. Configuration (car_hunter/config.py)

ROOT is the project folder (parent of car_hunter/). dotenv loads .env with override=True, so file values beat leftover shell vars.

Data filters

Constant	Value	Why
MIN_PRICE	$1,000	Drop junk / placeholder ads
MAX_PRICE	$80,000	Cap luxury outliers
MIN_YEAR	1995	Very old cars distort miles/price

MAX_ODOMETER	250,000	Drop broken odometer values
LITE_TRAIN/VAL/TEST	8000 / 1000 / 1000	Laptop-sized demo

Prompt constants (Week 7/8 specialist)

```
QUESTION = "What does this used car cost to the nearest dollar?"
PREFIX   = "Price is $"
```

make_prompt() builds: QUESTION + blank line + description + blank line + PREFIX + rounded price + '.00'. test_prompt() is the same string cut so the model must complete the number after PREFIX. export_sft.py writes the full prompt as JSONL {"text": ...}.

Environment variables

Variable	Default	Meaning
LLM_PROVIDER	ollama	ollama | groq | gemini | none/off
OLLAMA_MODEL	llama3.2	Local chat model name
GROQ_MODEL	llama-3.1-8b-instant	Groq free-tier model
GEMINI_MODEL	gemini-2.0-flash	Gemini model
GROQ_API_KEY	(empty)	Required if provider=groq
GOOGLE_API_KEY	(empty)	Gemini; GEMINI_API_KEY also accepted
SPECIALIST_ADAPTER	data/qlora_adapter	Folder with adapter_config.json
DEAL_THRESHOLD	1500	Min $ discount to alert

Index settings

- Embedding model: sentence-transformers/all-MiniLM-L6-v2 (384-d, fast CPU, good enough for listing text)
- Chroma path: data/chroma collection name: used_cars
- Artifact paths: train/val/test.parquet, random_forest.joblib, metrics.json, alerts.jsonl

6. Data pipeline (Week 6 curation)

Source

HF_DATASET = Yoad22/craigslist-used-cars-eda. License noted in the README as CC-BY-4.0, derived from Austin Reese's Kaggle Craigslist scrape. load_raw() uses datasets.load_dataset(..., split='train') then to_pandas(). Unauthenticated HF downloads work but are slower.

NEEDED columns

price, year, manufacturer, odometer, condition, fuel, transmission, drive, type, cylinders, paint_color, title_status, state, car_age. clean() keeps only columns that exist, coercing price/year/odometer to numeric,

dropping NA on price/year/manufacturer/odometer, applying the min/max filters, lowercasing make, dropping manufacturer == 'unknown', filling car_age = 2026 - year if missing, then drop_duplicates.

Split

train_test_split 80/20, then the 20% split 50/50 into val and test (so 80 / 10 / 10), random_state=42. If lite=True (default), each split is then sampled down to LITE_* caps with the same seed. prepare_data.py --full skips the lite sample. Saved as parquet without the index.

Why reconstruct listing text?
The cleaned public dataset dropped the original free-text ad and often the model name. listing_text_from_row rebuilds a readable paragraph from structured fields. That is the course 'LLM rewrite' step, without paying for a rewrite model. RAG and the specialist see this synthetic description, not the original Craigslist body.

listing_text_from_row

Always emits: '{year} {Make} used vehicle for sale. Mileage: {n} miles.' Then appends labeled facts when the value is present and not unknown/nan/none: Condition, Fuel, Transmission, Drivetrain, Body type, Cylinders, Color, Title status, Listed in {state}. Make is title-cased with underscores turned into spaces.

Re-running

```
uv run python scripts/prepare_data.py          # lite 8k/1k/1k
uv run python scripts/prepare_data.py --full   # full cleaned set
```

7. Domain models (car_hunter/listings.py)

CarListing (Pydantic)

title, manufacturer, year, price, odometer, description, optional prompt, condition, fuel, transmission, drive, body_type, state. listing_from_row sets title to '{year} {Make}', manufacturer lowercased, description from listing_text_from_row, then calls make_prompt().

Deal

listing + asking_price. In the scanner, asking_price is the listing's own price field (the historical ask).

Opportunity

deal + estimate + discount + notes. notes is a comma-separated string of agent guesses, e.g. random_forest=$17200, rag=$16840.

UI-only row quirks

The Estimate tab builds a fake row with price=0, transmission=automatic, cylinders/paint unknown, title_status=clean, car_age=2026-year. So the forest sees whatever make/condition you typed, but several fields are hardcoded. Extra text is appended to description and make_prompt() is called again.

8. Random Forest baseline (Week 6)

Features

CATEGORICAL: manufacturer, condition, fuel, transmission, drive, type, state. NUMERIC: year, odometer. Missing categoricals become the string 'unknown'. Note: car_age is computed in curation but is NOT a forest feature. Body type on the listing is body_type; in the dataframe column it is 'type'.

Pipeline

sklearn Pipeline: ColumnTransformer (OneHotEncoder handle_unknown='ignore' on cats + passthrough on nums) then RandomForestRegressor(n_estimators=120, random_state=42, n_jobs=-1, min_samples_leaf=2). min_samples_leaf=2 reduces tiny-leaf overfitting. handle_unknown='ignore' means a brand unseen in training contributes zeros for that dummy block instead of crashing.

Train vs predict

train_random_forest fits on all listings passed in (the train split) and dump()s to data/random_forest.joblib. predict_rf builds a one-row frame and clips negative predictions to 0. load_random_forest raises FileNotFoundError telling you to run train_baselines.py.

Median baseline

median_pricer ignores the listing and returns the median train price. It is a sanity floor: if RF cannot beat this, features are useless. On this machine's 400-row test eval: median MAE $9,135.70 vs RF MAE $3,970.55.

train_baselines.py flow

1. load_splits() -> listings_from_frame for train and test
2. train_random_forest(train)
3. evaluate_predictor(median, test, size=400)
4. evaluate_predictor(RF, test, size=400)
5. save_metrics to data/metrics.json

How to read the metrics

Metric	Median	Random Forest	Meaning
n	400	400	First 400 of test list, not a random 400
MAE	$9,136	$3,971	Average |guess - true price|
RMSE	$11,552	$5,548	Penalizes large misses more than MAE
R2	-0.028	0.763	Negative R2 = worse than predicting the mean

R2 of 0.76 means the forest explains a large share of price variance from year/miles/make/etc. MAE still ~$4k,

so 'deal' thresholds of $1,500 are inside the noise of the model. That is a key critique: some 'deals' are just model error, not true underpricing. Ensemble + RAG is meant to stabilize, not magically remove that error.

evaluate_predictor details

Takes listings[:size], runs predictor, parse_price on each guess (so LLM text 'about $12,500' becomes 12500.0), compares to item.price. running_mae exists for plots but the Gradio app does not use it.

9. Vector store and RAG (Week 8)

CarIndex

On init: SentenceTransformer(EMBEDDING_MODEL) and chromadb.PersistentClient(data/chroma), get_or_create_collection('used_cars'). Loading MiniLM from Hugging Face is why first startup is slow. Persistence means the second run should not re-embed if you do not call build().

build()

If the collection already has rows, it deletes and recreates it (full rebuild, not incremental). Batches of 256: encode descriptions, add ids as string offsets, documents=description, embeddings=float lists, metadatas={price, title, manufacturer}. Only TRAIN listings are indexed (build_index.py loads train_df only). That is correct: you must not put test rows in the index or RAG would leak the answer.

query()

Encodes the query description, collection.query n_results=5, returns (docs, prices). Prices come from metadata, not from parsing the document text.

RagAgent.price

1. Query 5 neighbors; if none, return 0.0
2. knn = mean of those 5 prices
3. If llm.available() is False, log 'No LLM; returning kNN mean' and return knn. THIS is the default path when Ollama is not running.
4. If an LLM is available, build a prompt: target description + each similar listing with its price + 'kNN mean of similars is $X'. System: 'Used-car pricing expert.' Reply with a number only.
5. parse_price the completion; on LLMUnavailable, fall back to knn.

Exam point
RAG here is retrieval + (optional) generation. Without an LLM it is still 'RAG-shaped' storage but the price is literally k-nearest-neighbor regression in embedding space. That is a valid, free specialist.

10. Agent system

Base Agent

car_hunter/agents/agent.py is logging plus ANSI colors (RED, GREEN, YELLOW, BLUE, MAGENTA, CYAN). log() writes logging.info with [Name] prefix. No tool-calling framework, no LangChain. 'Agent' here means a named component with a single responsibility.

ForestAgent

Loads RF once in __init__, price() -> predict_rf.

SpecialistAgent

If data/qlora_adapter/adapter_config.json exists, load base model from adapter_config['base_model_name_or_path'], wrap PeftModel, generate max_new_tokens=8, parse after PREFIX. Else if llm.available(), complete() with system 'You estimate used-car prices in US dollars. Reply with a number only.' Else return None so the ensemble drops the specialist.

QLoRA inference in specialist.py is NOT 4-bit; it is a full AutoModelForCausalLM + PeftModel. Fine-tuning was 4-bit on Colab; serving on a laptop without GPU may be heavy or fail. Ollama is the practical specialist on CPU.

EnsembleAgent

```
Always:  parts['random_forest'] = forest.price(listing)
Always:  parts['rag'] = rag.price(listing)
Maybe:   specialist = specialist.price(listing)
        if specialist is not None and specialist > 0:
            parts['specialist'] = specialist

If 3 parts:  0.5 * RF + 0.3 * RAG + 0.2 * Specialist
If 2 parts:  0.65 * RF + 0.35 * RAG
```

RF gets the largest weight because it is the strongest measured model on tabular features. RAG is a similarity smoother. Specialist is lowest weight because an unconstrained LLM can hallucinate a wild number. specialist == 0 is treated as missing (parse failure).

ScannerAgent

Loads test split, shuffles with random_state=7, keeps a cursor. scan(n) returns the next n rows as Deal objects, wrapping around to 0 at the end. So Hunt is a paging walk through 1,000 test ads, not a global search for the best deals. If you click Hunt twice, you get the next page, not the same 8.

PlanningAgent

score(deal): ensemble.price, discount = estimate - asking_price, notes from parts. hunt(n): for each scanned deal, score; if discount >= threshold, messenger.notify and collect; sort remaining by discount descending; log

how many beat the threshold.

MessengerAgent

Append one JSON object per line to data/alerts.jsonl: time (UTC ISO), title, asking, estimate, discount. Also logs a human sentence. It does not send email or SMS.

DealHunter

init_logging once, CarIndex, abort if collection.count()==0, PlanningAgent(index). estimate_listing delegates to planner.ensemble.price. hunt delegates to planner.hunt.

11. LLM layer (car_hunter/llm.py)

complete(prompt, system) switches on LLM_PROVIDER. available() is a cheap health check used by RagAgent and SpecialistAgent.

Provider	available() true when	complete() implementation
ollama	ollama.list() succeeds	ollama.chat(model, messages)
groq	GROQ_API_KEY set	groq.Groq chat.completions
gemini	GOOGLE_API_KEY or GEMINI_API_KEY	google.genai Client generate_content
none / off	False	raises LLMUnavailable

Default provider is ollama even if Ollama is not installed. available() catches exceptions and returns False, so forest+kNN still run. groq is imported only inside _groq(); it is not in pyproject.toml dependencies. Same for google.genai. You must pip/uv add those extra packages if you switch provider. ollama IS in pyproject.toml.

Optional QLoRA (Week 7)

1. uv run python scripts/export_sft.py -> data/train_sft.jsonl (and val/test)
2. Upload train_sft.jsonl to Colab (or Drive)
3. Open notebooks/qlora_colab.ipynb, runtime T4 GPU
4. Base model Qwen/Qwen2.5-1.5B-Instruct (ungated). Avoid Llama 3.2 if you do not want a Hugging Face license click-through.
5. 4-bit NF4, LoRA r=16 alpha=32 dropout=0.05 on q/k/v/o_proj, 1 epoch, max 4000 shuffled rows, max_length 256, batch 2, grad accum 8, lr 2e-4, paged_adamw_8bit
6. Download adapter into data/qlora_adapter/ so adapter_config.json is present, restart the app

12. Gradio UI (app.py)

Blocks with three tabs. Title: Used Car Deal Hunter. Launch (as last edited): share=True, server_name=127.0.0.1, server_port=7861. share=True tries to open a temporary gradio.live tunnel; on Windows that can hang while the local server is already healthy.

Estimate handler

Builds a dict row, listing_from_row, optional extra text, get_hunter().estimate_listing, returns (dollar string, breakdown lines, description). First click constructs DealHunter.

Hunt handler

Sets planner.threshold from the Number box, hunt(n), formats markdown bullets. Empty result tells you to lower threshold or scan more.

Metrics tab

Static markdown from metrics.json at demo-build time. Restart the process after retraining to refresh the table. It does not live-reload.

Default Estimate form values

2018 honda, 65,000 miles, good, gas, fwd, sedan, state ca. These are demo defaults, not a recommendation.

13. Scripts cheat sheet

```
cd car-deal-hunter
copy .env.example .env
uv sync
uv run python scripts\prepare_data.py
uv run python scripts\train_baselines.py
uv run python scripts\build_index.py
uv run python app.py

# optional specialist path
uv run python scripts\export_sft.py
# then Colab notebook -> data/qlora_adapter/
```

If data/ already contains parquet, joblib, and chroma, skip the three scripts. prepare_data downloads the HF dataset. train_baselines is ~40s of CPU forest. build_index is the slow step (~3 minutes here) because it embeds 8,000 documents.

Dependencies (pyproject.toml)

chromadb, datasets, gradio 5.x, joblib, numpy, ollama, pandas, pyarrow, pydantic, python-dotenv, scikit-learn, sentence-transformers. Hatchling packages the car_hunter directory. Requires Python >= 3.11.

14. End-to-end traces (walk these in your head)

Trace A: first Estimate click

1. get_hunter() -> DealHunter() -> CarIndex loads MiniLM + chroma; count must be > 0
2. PlanningAgent constructs Scanner (reads test.parquet, shuffles), Ensemble (loads RF joblib, RagAgent, Specialist which pings Ollama), Messenger
3. listing_from_row builds description; ensemble.price runs forest then rag (kNN unless LLM up) then specialist maybe None
4. Weighted sum returned to three textboxes

Trace B: Hunt deals

1. Same DealHunter singleton (scanner cursor already advanced if you hunted before)
2. scan(n) slices test rows, each becomes Deal(asking=historical price)
3. For each, ensemble again (forest + chroma query + optional LLM)
4. Keep discount >= threshold, append alerts.jsonl, sort by discount, render markdown

Trace C: Why first Hunt is slow

Each of n listings does a MiniLM encode + Chroma query + RF predict. n=8 is eight embedding queries. If Groq/Ollama is on, that is also eight (or sixteen, if both RAG and specialist call the LLM) network/local generations. Keep LLM_PROVIDER=ollama with Ollama stopped to stay on the fast kNN path.

15. Design decisions, limits, critique

Intentional

- Zero paid APIs: public dataset, sklearn, local embeddings, optional free LLM tiers.
- No live scrape: legal + reproducible demo.
- Lite 8k rows: course laptop, not production ML.
- Synthetic ads: dataset dropped free text; reconstruction is honest about that.
- Ensemble weights are hand-set, not learned on val. Val split is written but unused by train_baselines.py (only train fit + test eval). That is a gap you can mention in a write-up: val is ready for early stopping or weight search and currently idle.

Limitations to remember

- Test-set 'deals' are historical Craigslist asks vs a model, not verified market bargains.
- MAE ~$4k vs $1.5k threshold => false deals from noise.
- Scanner is sequential pages, not 'find the 10 most underpriced in the whole test set'.
- Estimate tab hardcodes transmission/title/cylinders/color.
- evaluate_predictor uses the first 400 test rows after listings_from_frame, not a shuffled 400 (train_baselines does not shuffle test).
- Chroma rebuild deletes the collection; interrupting build_index can leave an empty index.
- Specialist PEFT load may need GPU RAM; Colab trained 4-bit but local load path is not quantized.
- No leakage protection beyond 'index train only' -- if someone ran build_index on all data, metrics would

look fake.
- Gradio share is not a production deploy.

What 'good' looks like if you extend it

- Use val to tune ensemble weights or RF hyperparameters.
- Report MAE of ensemble, not only RF.
- Hunt the full test set and rank by discount / MAE-adjusted z-score.
- Calibration: plot predicted vs actual.
- Do not scrape live sites without ToS/legal review.

16. Study questions (with answers)

Q1. Why is the original ad text missing, and how does the project cope?

A: The public EDA dataset dropped free-text. listing_text_from_row rebuilds a templated description from structured columns so RAG/LLMs still have prose.

Q2. Which split goes into Chroma, and why?

A: Train only. Indexing test would leak neighbor prices that are the evaluation targets.

Q3. Write the two ensemble formulas.

A: Three agents: 0.5 RF + 0.3 RAG + 0.2 Specialist. Two agents: 0.65 RF + 0.35 RAG.

Q4. When does RAG skip the LLM?

A: When llm.available() is False (Ollama down, no Groq/Gemini key, provider none) or complete() raises LLMUnavailable. Then it returns the mean of 5 neighbor prices.

Q5. Define a deal.

A: Opportunity where estimate - asking_price >= PlanningAgent.threshold (env DEAL_THRESHOLD, default 1500).

Q6. Does Hunt search all 1,000 test cars?

A: No. It scans the next n rows of a shuffled feed (seed 7), then advances a cursor, wrapping around.

Q7. What does a negative R2 on the median baseline mean?

A: Predicting a constant median is slightly worse than predicting the mean of those 400 test prices; expected for a crude baseline. RF R2 0.76 is the real signal.

Q8. Name three things the project refuses to use.

A: OpenAI fine-tuning/GPT APIs, Modal GPU hosting, live marketplace scraping.

Q9. Where are alerts stored?

A: data/alerts.jsonl, one JSON object per deal, appended, UTC timestamp.

Q10. Why might DealHunter raise FileNotFoundError at startup?

A: Chroma collection count is 0 -- you skipped or failed build_index.py. ForestAgent can separately fail if random_forest.joblib is missing.

Q11. What is the SFT target string shape?

A: QUESTION, description, PREFIX+price.00 so a causal LM learns to emit a dollar amount after 'Price is $'.

Q12. Why is val.parquet unused at train time?

A: train_baselines fits on train and evaluates on test (400 rows). Val is saved for a proper ML workflow you could add (tuning). Mention this as a course-style improvement.

17. Glossary

Term	In this project
MAE	Mean |predicted price - true price| in dollars
RMSE	Sqrt of mean squared error; big mistakes hurt more
R2	1 - SS_res/SS_tot; 1 is perfect, 0 is 'predict the mean'
RAG	Retrieve similar listings from Chroma; optionally generate a price with an LLM
kNN price	Average price of the k nearest embedding neighbors (k=5)
QLoRA	Quantized LoRA fine-tune of a small instruct model on Colab T4
Ensemble	Weighted average of agent prices
Deal / Opportunity	Test listing with large enough estimated discount
Lite split	8k/1k/1k sample after 80/10/10 split
Scanner feed	Shuffled test.parquet, not the internet
Specialist	QLoRA adapter or generic LLM completing the price prompt
MiniLM	all-MiniLM-L6-v2 sentence embedding model

18. File-by-file one-liners

- config.py -- single source of paths, filters, env.
- curator.py -- HF download, clean, split, load_splits.
- listings.py -- CarListing/Deal/Opportunity + text templates.
- baselines.py -- OHE+RF pipeline, median dummy, joblib I/O.
- evaluator.py -- parse_price regex, sklearn metrics, JSON save.
- vectorstore.py -- MiniLM + Chroma persist/query.
- llm.py -- provider switch and available().
- framework.py -- DealHunter facade + logging.
- agents/agent.py -- colored logger base class.
- agents/forest.py -- wrap RF.

- agents/rag.py -- kNN or LLM-over-neighbors.
- agents/specialist.py -- PEFT or LLM or None.
- agents/ensemble.py -- 2- or 3-way weighted blend.
- agents/scanner.py -- paging test-set feed.
- agents/planner.py -- score + threshold + sort.
- agents/messenger.py -- JSONL alerts.
- agents/__init__.py -- exports EnsembleAgent, PlanningAgent.
- app.py -- Gradio.
- scripts/*.py -- CLI entry points.
- qlora_colab.ipynb -- Week 7 training recipe.

19. Commands and local artifacts after a successful setup

After prepare + train + index you should see approximately:

- data/train.parquet, val.parquet, test.parquet
- data/random_forest.joblib
- data/metrics.json (median + random_forest rows)
- data/chroma/ (Chroma sqlite + embeddings, 8,000 ids)
- optional: data/alerts.jsonl after you Hunt
- optional: data/train_sft.jsonl after export_sft.py

Local UI: http://127.0.0.1:7861 (port as configured in app.py). Forest test MAE about $3,971; median about $9,136; RF R2 about 0.763 on 400 listings -- numbers will change if you retrain with --full or another seed.

End of study guide. If you can explain the architecture diagram, the ensemble formulas, why train-only indexing matters, and why a $1,500 deal threshold is aggressive given ~$4k MAE, you understand the project.